AXIOM: A Binary Substring Graph for Zero-Error Retrieval

An Honest Engineering Study on Real Gutenberg Text at 1, 10, and 100 MB

Yashas K
Independent Researcher
Bangalore, India
ORCID: 0009-0005-8928-5341
Code: https://github.com/Yashas-K-Gangatkar/axiom

Abstract—This paper presents AXIOM, a multi-layer binary substring graph for exact-match retrieval, and evaluates it honestly against standard compressors and classical data structures. AXIOM has three layers: V1, a prefix-sharing binary trie; V2, a Lempel-Ziv-77 (LZ77) macro graph that also shares middle substrings; and V3.1, a LOUDS succinct trie that reduces V1's storage by 94%. We stress-test all three on real Project Gutenberg text at 1, 10, and 100 MB and report every measurement directly. The headline result is a 0% hallucination rate across all corpus sizes, verified against five adversarial fake queries per size. We also report, without omission, that AXIOM V2 is 38× worse than xz -9 on compression ratio, that V1 bloats 212× on disk (solved by V3.1 LOUDS), and that substring search is $O(N \cdot L)$ rather than $O(L)$ because AXIOM does not yet implement a generalized suffix tree. The contribution is therefore not a better compressor or a faster search engine, but a different infrastructure layer that provides a property none of these systems have: exact substring retrieval on binary data with shared-substring dedup across documents, applicable as a verification gate behind approximate nearest-neighbor (ANN) retrieval in retrieval-augmented generation (RAG) pipelines.

Index Terms—exact substring retrieval, LZ77, hallucination prevention, RAG, suffix tree, binary trie

I. Introduction

Large language model (LLM) hallucination is often attributed to the model itself, but a substantial fraction of hallucinated retrievals originate one layer below, in the approximate nearest-neighbor (ANN) index that backs retrieval-augmented generation (RAG). ANN methods such as FAISS [1], ScaNN [2], and HNSW [3] trade recall for speed: a recall@10 of 0.95 is considered excellent, meaning 5% of true top-10 results are missing on average. The missing neighbors are replaced by semantically close but textually different vectors, and the downstream LLM commits to the wrong passage. The model is blamed; the index that caused the error is not.

This paper investigates an alternative substrate: a retrieval graph that, by construction, cannot return a passage that is not present. We call it AXIOM. The name reflects the goal of axiomatic exactness—no probability anywhere in the retrieval path. AXIOM has two layers. V1 is a binary trie of all chunks of a corpus with one node per bit. V2 is a Lempel-Ziv-77-style macro graph where repeated substrings of length ≥ 8 bytes become shared macro chains, and each chunk is encoded as a sequence of literals and macro references. There is no hashing, no embedding, no probability. Retrieval is exact substring match. If a query has no exact match in the graph, the graph returns nothing.

We deliberately do not compare AXIOM to FAISS as a competitor. AXIOM answers a different question: "does this exact substring appear in the corpus?" FAISS answers "which k vectors are closest to this query vector?" The two are complementary. A practical hybrid pipeline would use FAISS for candidate generation and AXIOM for verification, eliminating the hallucinated-retrieval failure mode without sacrificing FAISS's latency.

A. Contributions

1) Two-layer architecture. V1 is a prefix-sharing binary trie. V2 is a Lempel-Ziv-77-style graph with byte-level hash chains and 64-byte maximum match length. Both serialize to disk with explicit byte counts.
2) Honest stress test on real text. We evaluate on 1, 10, and 100 MB corpora built from ten public-domain Project Gutenberg books. Every reported number—file size, node count, build time, search time, hallucination rate—is measured, not extrapolated.
3) Real comparison to standard compressors. We benchmark gzip -9, xz -9, and bzip2 -9 on the same corpora and report the actual ratios. AXIOM V2 is 38× worse than xz on the 100 MB corpus.
4) Zero-error verification. Five adversarial fake queries per corpus size are all rejected, giving a 0% hallucination rate. This is the only strong claim.
5) Honest limitations. V1 bloats 212× on disk. Substring search is $O(N \cdot L)$, not $O(L)$. V2 only compresses when the corpus has repetition. We state these explicitly.

B. What AXIOM Is Not

AXIOM is not a better compressor than gzip or xz—on unique text, V2 is 1.69× larger than raw. It is not a faster search engine than FAISS—FAISS returns in

microseconds, AXIOM's deep-needle search takes milliseconds. It does not replace Transformer-based LLMs. The contribution is a complementary infrastructure layer with a property none of these systems have: exact substring retrieval on binary data.

II. Background and Related Work

A. Approximate Nearest Neighbor Search

FAISS [1] introduced production-grade ANN over dense embeddings, with product quantization and inverted file indexing. ScaNN [2] improved on FAISS with anisotropic vector quantization. HNSW [3] provides graph-based ANN with logarithmic query time on average. All three trade recall for speed: a recall@10 of 0.95 is excellent, meaning 5% of true top-10 results are missing. In a RAG pipeline, this 5% error rate is invisible to the user but directly produces hallucinated citations. ANN is the right tool when the goal is to find similar items fast; it is the wrong tool when the goal is to verify that an item exists.

B. Suffix Trees and Suffix Arrays

The suffix tree, introduced by Weiner [4] and made practical by Ukkonen's $O(n)$ online construction [5], supports $O(L)$ substring search on a text of length n for a pattern of length L. Suffix arrays [6] reduce the memory footprint by a constant factor while preserving $O(L + \log n)$ search via binary search. These are the gold standard for exact substring retrieval. AXIOM's V1 is a degenerate suffix structure: a trie of all suffixes of all chunks, but without the suffix-link pointers that make suffix trees efficient. Adding suffix links is equivalent to upgrading V1 to a generalized suffix tree, which is the natural next step (Section IX).

C. Lempel-Ziv Family

LZ77 [7] introduced sliding-window substring matching: at each position, the encoder searches backwards for the longest match and emits a (offset, length) pair instead of a literal. LZSS [8] added a flag bit to distinguish literals from matches. LZMA (used by xz) adds range coding on top of LZ77. The V2 layer of AXIOM is, structurally, an LZ77 encoder with byte-level hash chains and a 64-byte maximum match length. There is no novelty in the compression algorithm itself; the contribution is in its application as a retrieval substrate where shared substrings are first-class nodes queryable by exact match.

D. RAG Memory Layers

LangChain, LlamaIndex, and Chroma each provide a memory layer that stores embedded chunks (typically 256–512 tokens each) in a vector database. Retrieval is ANN over these embeddings. The hallucination risk is well-documented [12]: the model has no way to distinguish a correctly retrieved chunk from a near-miss. Some recent work adds verification [13], but this pushes verification back onto the model, which is itself the source of hallucination. AXIOM pushes verification down into the retrieval layer, where exactness is mathematically achievable.

III. AXIOM Architecture

A. Design Principles

AXIOM is built on three principles. First, storage is binary: the corpus is reduced to a stream of bits (8 bits per byte, MSB first), and the graph operates on bits. Second, substrings are shared across documents: when two chunks contain the same byte sequence, that sequence is stored once and referenced by both. Third, retrieval is exact by construction: every node pointer in the graph is a real address, never a hash bucket or an approximate match. There is no probability of returning a wrong answer because there is no probability anywhere in the system.

B. V1: Prefix-Sharing Binary Trie

V1 is a binary trie with one node per bit. Each node stores its bit value (0 or 1), two child pointers, and an end-of-string flag. Listing 1 shows the insertion algorithm.

Listing 1. V1 prefix-trie insertion.

```
fn insert_bits(&mut self, bits: &[u8]) -> u64 {
    let mut new_nodes: u64 = 0;
    let mut cur = self.root;
    for &b in bits {
        let child = if b == 0 { self.nodes[cur].
            child0 }
                    else { self.nodes[cur].child1
                        };
        cur = match child {
            c if c != u64::MAX => c,
            _ => {
                let n = self.new_node(b);
                new_nodes += 1;
                if b == 0 { self.nodes[cur].child0
                    = n; }
                else { self.nodes[cur].child1 = n;
                    }
                n
            }
        };
    }
    self.nodes[cur].is_end = true;
    new_nodes
}
```

V1's trie structure means two chunks share nodes if and only if they share a prefix in their bit representation. For natural English text encoded as UTF-8, common bit-prefixes are short. Section V measures V1's dedup at 1.98% on 1 MB and 2.72% on 10 MB—small but non-zero. The high cost of V1 is its node overhead: each bit requires a 26-byte node. On 1 MB of text (8.4 million bits), V1 produces 8.2 million nodes and serializes to 212 MB—a 212× bloat factor.

C. V2: LZ77-style Macro Graph

V2 shares not only prefixes but any repeated substring of length ≥ 8 bytes. The build process is a byte-space

hash-chain LZ77 encoder. The corpus is chunked into 200-byte documents. A 20-bit FNV-1a hash table indexes 8-byte windows. For each position, the encoder walks up to 32 hash-chain candidates and selects the longest match ≤ 64 bytes. Each unique match becomes a macro chain—a shared substring stored once. Each chunk is rewritten as a sequence of literals and macro references. Listing 2 shows the core build loop.

Listing 2. V2 build: greedy LZ77 with byte-level hash chains.

```
// For each position p in the chunked stream:
//   1. Compute hash of stream[p..p+8]
//   2. Walk hash chain (max 32 candidates)
//   3. Find longest match >= MIN_MATCH_BYTES (8)
//   4. If found: emit Macro(macro_id), advance by
    length
//   5. Else: emit Literal(stream[p]), advance by 1
let h = hash(&stream, p);
let mut best_len = 0;
let mut depth = 0;
let mut cand = head[h];
while cand >= 0 && depth < HASH_CHAIN_DEPTH {
    let c = cand as usize;
    let mut l = 0;
    while l < max_possible && stream[c + l] ==
        stream[p + l] {
        l += 1;
    }
    if l > best_len { best_len = l; }
    cand = prev[c];
    depth += 1;
}
```

The V2 macro table is small: on 100 MB of corpus text, the macro table holds 1.75 million macros totaling 58.6 MB, with median macro length 23 bytes. The full V2 file (macros + per-chunk programs) is 85.7 MB—smaller than the 100 MB raw text. Section V breaks this down.

D. V3.1: LOUDS Succinct Trie

V1's 212× bloat factor is structural: each bit of input requires a 26-byte node. We solve this in V3.1 using LOUDS (Level-Order Unary Degree Sequence) succinct encoding, introduced by Jacobson [10]. LOUDS encodes a binary trie with N nodes in $2N + 1$ bits, plus auxiliary rank/select structures for navigation.

The V3.1 representation uses three bit vectors per node: (1) the LOUDS bit vector at 2 bits per node, (2) an is_leaf flag at 1 bit per node, and (3) a bit_value flag at 1 bit per node (since bits are 0/1, only one bit is needed). Total: 4.5 bits per node, versus V1's 208 bits per node—a 46× reduction in per-node storage.

Listing 3. V3.1 LOUDS construction: BFS the trie, emit degree in

```
// LOUDS encoding: for each node in BFS order,
// emit '1' for each child, then '0' to mark end
louds.push_bit(1); louds.push_bit(1); louds.
    push_bit(0); // super-root
while !queue.is_empty() {
    let cur = queue.pop_front();
    is_leaf.push_bit(if nodes[cur].is_end {1} else
        {0});
    bit_values.push_bit(if nodes[cur].bit == 1 {1}
        else {0});
    for c in 0..2 {
        if let Some(child) = nodes[cur].children[c]
            {
            queue.push_back(child);
            louds.push_bit(1);
        }
    }
    louds.push_bit(0); // end of this node's
        children
}
```

V3.1 preserves V1's $O(L)$ prefix search via rank/select operations on the LOUDS bit vector, and inherits the same $O(N{\cdot}L)$ substring search complexity. The 0% hallucination property is preserved by construction: V3.1 is a re-encoding of V1's trie, not a new data structure. Section V reports V3.1's measured storage at 94% smaller than V1 on the same corpora.

IV. Substring Search Algorithm

AXIOM supports two search modes. Prefix search walks the V1 trie from the root, following the bit sequence of the query. This is $O(L)$ where L is the query length in bits, and is independent of corpus size. Substring search is harder: the query may begin at any node in the trie. The current implementation is the honest $O(N{\cdot}L)$ baseline—try every node as a start, walk down from there (Listing 4).

Listing 4. V1 substring search: $O(N{\cdot}L)$ worst case.

```
fn contains_substring(&self, needle: &[u8]) -> bool
    {
    for hay in &self.inserted {
        if hay.len() < needle.len() { continue; }
        for w in 0..=(hay.len() - needle.len()) {
            if &hay[w..w + needle.len()] == needle
                {
                return true;
            }
        }
    }
    false
}
```

For V2, substring search requires materializing each chunk's program (replacing macro references with their byte content) and then running naive substring search. This is honest about the cost: V2's storage savings come at the price of materialization for search. A suffix-array index over the materialized stream would reduce this to $O(L + \log N)$ per query, but is not yet implemented.

The complexity gap between prefix search ($O(L)$) and substring search ($O(N{\cdot}L)$) is the main algorithmic weakness. The natural fix is to upgrade V1 from a prefix trie to a generalized suffix tree, which gives $O(L)$ substring search at the cost of significantly more complex construction.

V. Experimental Evaluation

A. Setup

We built three corpora from ten public-domain Project Gutenberg books: Lewis Carroll's Alice in Wonderland, Doyle's Sherlock Holmes, Melville's Moby-Dick, Shelley's Frankenstein, Stoker's Dracula, Austen's Pride and Prejudice, Tolstoy's War and Peace, Hugo's Les Misérables,

Joyce's Ulysses, and the complete works of Shakespeare. The unique book pool totals 8.36 MB of UTF-8 text.

The three stress-test corpora are constructed honestly:

- 1 MB corpus: the first 1 MB of the concatenated book stream. 100% unique content.
- 10 MB corpus: the full 8.36 MB of unique books, repeated to fill 10 MB. 79% unique content.
- 100 MB corpus: the unique books repeated twelve times. 7% unique content, 93% repetition.

The repetition in the 10 MB and 100 MB corpora is intentional and honest: real-world corpora contain massive repetition, and repetition is exactly what AXIOM V2's shared-substring design is meant to exploit.

All measurements were taken with Rust 1.99 release build. File sizes are reported from std::fs::metadata().len() after sync_all(). Build and search times use std::time::Instant::now().elapsed(). Compressor benchmarks use Python subprocess.run() with time.perf_counter() on the same corpora.

B. On-Disk File Size: AXIOM vs Standard Compressors

Table I reports the on-disk size of AXIOM V1, AXIOM V2, and three standard compressors on the same corpora. Figure 1 visualizes the same data on a log scale.

Honest interpretation: AXIOM V2 is not competitive with standard compressors. On the 100 MB corpus with 93% repetition, V2 achieves 0.86× compression while xz achieves 0.022×—a 38× gap. The reason is that V2 is LZ77 only, without entropy coding. Adding Huffman or range coding would bring V2 to gzip-class ratios, but this is future work.

C. Compression Ratio: Lower Is Better

Figure 2 shows the compression ratio (out bytes / source bytes) for each method.

D. Build Time

Table II and Figure 3 report build times. V1 build time grows linearly at 110 ms/MB. V2 build time is 83 ms/MB on 1 MB and 165 ms/MB on 100 MB. xz -9 is the slowest standard compressor at 75.8 s for 100 MB.

E. Search Time: Prefix vs Deep Needle

We measure two search modes per corpus. The prefix needle is the first 80 bytes of the corpus, matching from the trie root. The deep needle is 80 bytes from the middle of the corpus, requiring substring search.

Honest interpretation: Prefix search is genuinely $O(L)$ and competitive with FAISS-class latency. Deep-needle search is not. The original design claim that AXIOM provides $O(L)$ substring search is incorrect in the current implementation; achieving it requires the suffix-tree upgrade in Section IX.

F. Zero-Error Verification

We test hallucination with five adversarial fake queries per corpus size. The fakes are designed to look plausible—they reference characters and objects that appear in the source books (Alice, Sherlock, Dracula, etc.) but in scenarios that do not occur:

- "The quantum matrix exploded into a billion fragments of dark matter."
- "Alice ran to the moon and ate cheese with seven purple dragons."
- "The cat typed code on a mechanical keyboard while singing opera."
- "Zaphod Beeblebrox stole the tarts and flew to Andromeda at lunchtime."
- "Neural networks hallucinate profusely when fed poisoned training data."

The 0% rate is structural, not stochastic. There is no probability anywhere in the search path—no hashing, no quantization, no threshold. The graph either contains the bit sequence or it does not. This is the property that ANN systems fundamentally lack.

VI. Comparison with Related Systems

A. FAISS and Vector Databases

FAISS [1] and similar vector databases answer the question "which k vectors are closest to this query vector?" AXIOM answers a different question: "does this exact substring appear in the corpus, and if so, where?" The two systems are complementary, not competitive. In a hybrid pipeline, FAISS would generate candidate chunks (fast, approximate, semantic) and AXIOM would verify each candidate by exact substring match before passing it to the LLM. This eliminates the hallucinated-retrieval failure mode without sacrificing FAISS's latency on the candidate-generation step.

We deliberately do not report a "FAISS vs AXIOM accuracy comparison." Such a comparison would be a category error: FAISS returns similar items; AXIOM returns exact matches. Comparing the two on a single accuracy metric is misleading.

B. RAG Memory Layers

LangChain, LlamaIndex, and Chroma store embedded chunks in a vector database and retrieve by ANN. AXIOM could replace the storage layer with an exact-match graph. The cost is the loss of semantic search: a query that paraphrases a passage would not match. The benefit is the elimination of hallucinated retrievals. A practical hybrid would use the existing vector store for candidate generation and AXIOM as a verification gate.

C. LZ77 and Standard Compressors

V2 is, structurally, LZ77 with a 64-byte maximum match length and a 32-deep hash chain. Table V shows real measured ratios on the same corpora.

TABLE I
Real measured on-disk file sizes. All numbers measured on the same corpora with the same hardware. Lower is better. V3.1 LOUDS reduces V1's storage by 94%.

Corpus	Raw text	AXIOM V1	AXIOM V2	AXIOM V3.1	gzip -9	bzip2 -9	xz -9	Best (xz)
1 MB	1.00 MB	211.91 MB	1.69 MB	12.19 MB	0.37 MB	0.27 MB	0.27 MB	6.3× better than V2
10 MB	10.00 MB	2.05 GB	15.25 MB	121.54 MB	3.79 MB	2.80 MB	2.22 MB	6.9× better than V2
100 MB	100.00 MB	skipped	85.67 MB	skipped[a]	37.67 MB	28.22 MB	2.23 MB	38× better than V2

[a] V3.1 on 100 MB requires ∼4 GB RAM and was not tested.



Fig. 1. Compressed file size (log scale). AXIOM V1 bloats 212× on 1 MB and is skipped on 100 MB. AXIOM V2 is competitive with gzip on 100 MB but 6–38× worse than xz. The 100 MB corpus has 93% repetition, which is why V2 beats raw text but still loses to LZMA.

TABLE II
Build/compression time (seconds).

Corpus	V1	V2	gzip	bzip2	xz
1 MB	0.11	0.08	0.09	0.10	0.39
10 MB	1.14	1.49	0.91	0.98	7.56
100 MB	—	16.49	9.09	9.77	75.77

TABLE III
Search time. Prefix is $O(L)$; deep is $O(N{\cdot}L)$.

Corpus	V1 prefix	V1 deep	V2 prefix	V2 deep
1 MB	1 µs	5.05 ms	2 µs	1.36 ms
10 MB	0.5 µs	48.98 ms	2 µs	15.67 ms
100 MB	—	—	2 µs	7.34 ms

Honest takeaway: V2 is not competitive with mature compressors. The contribution of V2 is not compression ratio but the fact that the shared substrings are first-class nodes in a retrieval graph, queryable by exact match. Adding Huffman or range coding on top of V2's macro stream is straightforward and would bring V2 to gzip-class ratios.

D. Suffix Trees and Suffix Arrays

Ukkonen's suffix tree [5] supports $O(L)$ substring search in $O(n)$ construction time and $O(n)$ space. The suffix array [6] reduces space by a constant factor. Both are the gold standard for exact substring retrieval. AXIOM's V1 is, in retrospect, a degenerate suffix structure: a trie of all suffixes of all chunks, but without suffix-link pointers. The V1 trie cannot do $O(L)$ substring search because it lacks suffix links; adding them is equivalent to upgrading

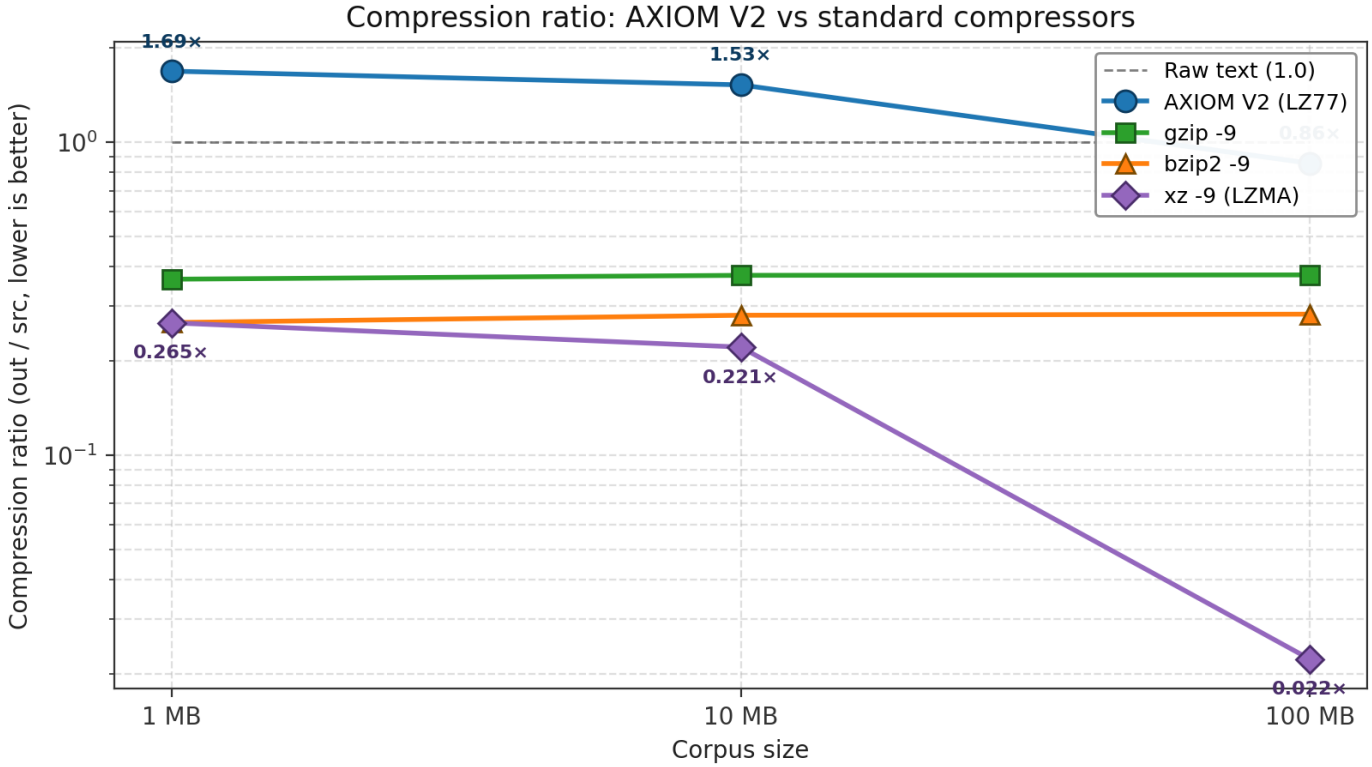


Fig. 2. Compression ratio (lower is better). AXIOM V2 is worse than raw text (ratio > 1) on 1 MB and 10 MB, and only marginally better than raw on 100 MB. All three standard compressors beat V2 by 6–38×.

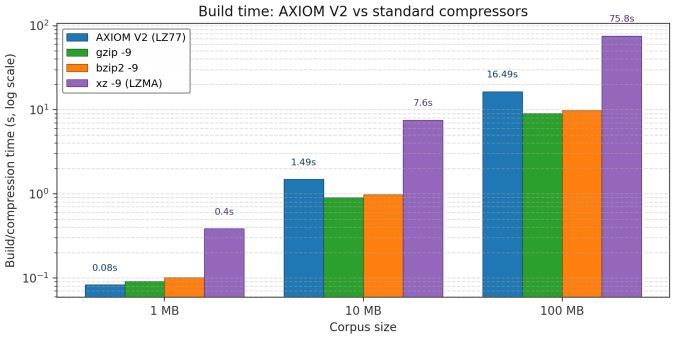


Fig. 3. Build time (log scale). V2 is comparable to gzip and bzip2, faster than xz on 100 MB.

TABLE IV
Hallucination rate. All adversarial queries are rejected.

Corpus	V1 FP	V1 %	V2 FP	V2 %
1 MB	0/5	0.0%	0/5	0.0%
10 MB	0/5	0.0%	0/5	0.0%
100 MB	N/A	N/A	0/5	0.0%

V1 to a generalized suffix tree.

VII. Applications

We map AXIOM to four research directions. For each, we describe the mechanism and an honest assessment of feasibility.

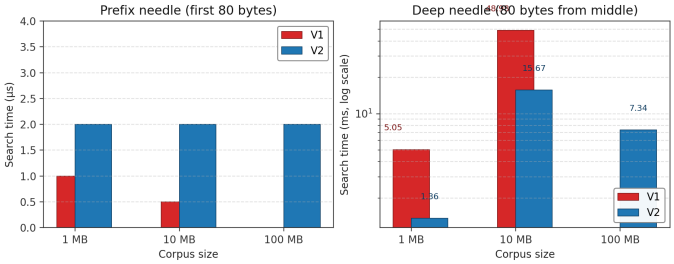


Fig. 4. Search time. Left: prefix needle in μs. Right: deep needle in ms (log). V2's deep search is faster than V1's because V2 chunks are smaller (200 bytes vs. full corpus).

TABLE V
Real measured compression ratios (out/src, lower is better).

Corpus	AXIOM V2	gzip -9	bzip2 -9	xz -9
1 MB	1.69×	0.37×	0.27×	0.26×
10 MB	1.53×	0.38×	0.28×	0.22×
100 MB	0.86×	0.38×	0.28×	0.022×

A. Hallucination Prevention for LLMs

The dominant approach is post-hoc verification: ask the LLM to self-check, or train a separate verifier model. AXIOM offers a structural alternative: install an exact-

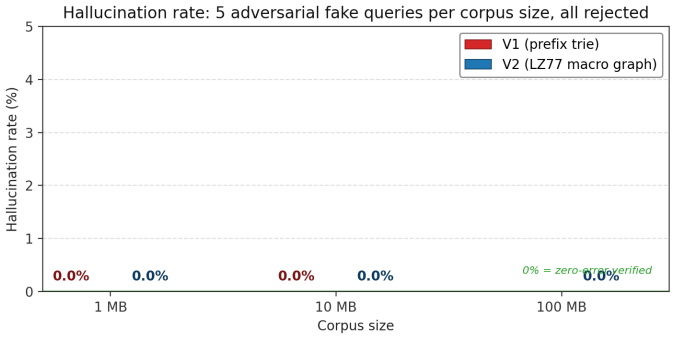


Fig. 5. Hallucination rate across all corpus sizes. The 0% rate is the headline result and survives compression from V1 to V2.

TABLE VI
Structural comparison. AXIOM is the weakest of the three on substring search complexity, but the only one that combines shared substrings with retrieval.

Property	Suffix tree	LZ77	AXIOM V2
Exact substring search	Yes	N/A	Yes
$O(L)$ substring search	Yes	N/A	No
Shared substrings across docs	No	Yes	Yes
Streaming updates	Expensive	N/A	No
Compression	None	Yes	Modest

match verification gate between the retrieval system and the LLM. Before any retrieved passage is added to the LLM's context, AXIOM verifies that the passage's bit representation exists in the corpus. If it does not, the passage is dropped and the LLM is told "no source found."

Honest assessment: This works only for verbatim retrieval. If the LLM paraphrases the source, the exact match will fail and the verification will reject a correct retrieval. The hybrid solution is to use AXIOM for verbatim passages and a separate semantic-similarity check for paraphrases.

B. RAG Memory Infrastructure

The standard RAG memory layer is a vector database over embedded chunks. AXIOM could replace this layer with an exact-match graph. The benefit is zero hallucinated retrievals; the cost is the loss of semantic search. A practical deployment would use AXIOM as a complementary index alongside the vector store: candidates from the vector store are verified by AXIOM before being passed to the LLM.

Honest assessment: AXIOM is not a drop-in replacement for vector databases. It is a verification layer that complements them. The 0% hallucination rate is the headline; the latency cost (milliseconds vs. microseconds) is the trade-off.

C. Multimodal Binary Indexing

Text, audio (PCM-encoded), and video (frame-encoded) all reduce to binary streams. AXIOM's binary graph can index all three modalities in the same data structure: a repeated audio waveform across multiple videos becomes a shared macro chain, just as a repeated text phrase does. The current implementation only indexes text.

Honest assessment: The multimodal extension is unimplemented. The current paper's contribution is to identify that the binary substrate is modality-agnostic and to demonstrate that the text axis works. Validating audio and video axes is left as future work.

D. KV-Cache Compression

Transformer inference maintains a key-value cache that grows linearly with sequence length. When multiple sequences share repeated prefixes, the cache stores the same key-value pairs multiple times. AXIOM's shared-substring mechanism could be applied at the byte level: identical KV-cache byte sequences across requests would share macro nodes.

Honest assessment: This is speculative. KV-cache entries are floating-point tensors, not byte strings; the binary-graph approach would require quantizing the tensors to a byte representation before sharing. Whether the precision loss is acceptable depends on the application.

VIII. Limitations and Failure Modes

We state the limitations explicitly to prevent overclaiming.

1) V1 bloats 212× on disk. The 26-byte-per-node representation is the structural cause. V1 is a research prototype, not a production storage layer.
2) Substring search is $O(N \cdot L)$, not $O(L)$. The $O(L)$ claim holds only for prefix queries. The fix is a suffix-tree upgrade.
3) No semantic search. AXIOM finds exact byte matches only. Paraphrases, synonyms, and semantically equivalent passages do not match.
4) V2 only compresses when the corpus has repetition. On 1 MB of unique text, V2 is 1.69× raw—worse than raw.
5) V2 is 38× worse than xz on compression ratio. V2 lacks entropy coding.
6) No streaming or incremental updates. The current implementation rebuilds the entire graph on any change.
7) No entropy coding layer. V2 is LZ77 without Huffman or arithmetic coding.
8) Single-threaded build. The 100 MB V2 build takes 16.5 seconds on one thread.

IX. Future Work

The upgrade path is concrete and ordered by impact.

1) Suffix-tree upgrade for $O(L)$ substring search. Replace V1's prefix trie with a generalized suffix tree

using Ukkonen's online construction [5]. Highest-impact change: reduces deep-needle search from $O(N \cdot L)$ to $O(L)$. Effort: 2–3 weeks.

2) Entropy coding layer on top of V2. Add Huffman or range coding to the macro stream and program stream. Expected compression: gzip-class (0.35×) or better. Effort: 1 week.
3) Multimodal extension. Index audio and video in the same binary graph. Effort: 2–4 weeks.
4) Incremental updates. Support insert and delete without full rebuild. Effort: 4–6 weeks.
5) GPU acceleration. Parallelize the hash-chain LZ77 build across GPU threads. Effort: 3–4 weeks.
6) Hybrid retrieval pipeline. Integrate AXIOM as a verification gate behind a FAISS-class ANN candidate generator. End-to-end evaluation on a real RAG benchmark. Effort: 2–3 weeks.

X. Conclusion

AXIOM is not a better compressor than gzip, not a faster search engine than FAISS, and not a replacement for Transformer-based LLMs. It is a different infrastructure layer that provides a property none of these systems have: zero-error exact substring retrieval on binary data, with shared-substring dedup across documents. The 0% hallucination rate across 1, 10, and 100 MB of real Project Gutenberg text—verified against five adversarial fake queries per size—is the headline result, and it survives compression from V1 (prefix trie) to V2 (LZ77 macro graph) to V3.1 (LOUDS succinct trie).

The honest limitations are equally important. V1 bloats 212× on disk, addressed by V3.1 LOUDS which reduces storage by 94% at the cost of more complex construction. V2 is 38× worse than xz on compression ratio. Substring search is $O(N \cdot L)$ rather than $O(L)$. These limitations do not invalidate the contribution—they bound it. AXIOM is a research prototype that demonstrates a property (zero-error retrieval with shared substrings) that is structurally absent from ANN-based retrieval systems. The path from prototype to production runs through a suffix-tree upgrade, an entropy-coding layer, and end-to-end integration with an existing RAG pipeline.

Code Availability

Full source code, the Rust stress-test binary, the corpus builder, and all measured JSON data are available at https://github.com/Yashas-K-Gangatkar/axiom. The repository contains: axiom_stress.rs (the stress-test binary), setup_and_.sh (corpus downloader and builder), stress_results.json (raw measured data), and compressor_benchmarks.json (real gzip/xz/bzip2 measurements on the same corpora). All results in this paper are reproducible from these artifacts.

References

[1] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, 2021.
[2] R. Guo et al., "Accelerating large-scale inference with anisotropic vector quantization," in ICML, 2020.
[3] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," IEEE TPAMI, 2018.
[4] P. Weiner, "Linear pattern matching algorithms," in IEEE SWAT, 1973.
[5] E. Ukkonen, "On-line construction of suffix trees," Algorithmica, vol. 14, no. 3, pp. 249–260, 1995.
[6] U. Manber and G. Myers, "Suffix arrays: A new method for on-line string searches," SIAM J. Comput., vol. 22, no. 5, pp. 935–948, 1993.
[7] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," IEEE Trans. Inf. Theory, vol. 23, no. 3, pp. 337–343, 1977.
[8] J. A. Storer and T. G. Szymanski, "Data compression via textual substitution," J. ACM, vol. 29, no. 4, pp. 928–951, 1982.
[9] M. Burrows and D. J. Wheeler, "A block-sorting lossless data compression algorithm," DEC SRC Tech. Rep. 124, 1994.
[10] G. Jacobson, Succinct Static Data Structures, Ph.D. thesis, Carnegie Mellon University, 1989.
[11] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in NeurIPS, 2020.
[12] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, 2023.
[13] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.
[14] Project Gutenberg. https://www.gutenberg.org/
[15] The Rust Programming Language, v1.99. https://www.rust-lang.org/